

category: react-web

tags: [web-speech-api, speechSynthesis, tts, promise-chain, vanilla-js, module-export]

severity: high

created: 2026-07-31

updated: 2026-07-31

project-origin: EnglishTool (LangTool)

Hàm mới thêm vào module IIFE quên đưa vào ``return {...}`` → gọi qua ``U.fn()`` throw `TypeError`, giết im lặng phần còn lại của promise chain TTS

Tình huống gặp phải

| Đang làm gì? Tính năng gì? Môi trường nào?

Thêm tính năng đọc nối tiếp nhiều đoạn bằng Web Speech API (`speechSynthesis`) trong chế độ học từ vựng "Focus Mode": mỗi lần đổi từ, đọc theo chuỗi **từ → nghĩa (tiếng Việt) → đánh vần từng chữ cái → ví dụ**, implement bằng cách nối các lệnh gọi `speak()` (mỗi lệnh trả về 1 Promise resolve khi `onend`) qua `.then()`. Thêm hàm mới `speakSpelled()` vào module `window.U = (function(){...})()` (IIFE, không phải ES module) để đánh vần từng chữ cái.

Triệu chứng / Lỗi

| Lỗi hiện ra như thế nào? Exception message? Behavior sai?

Không có exception nào **hiển thị cho user** (chỉ có trong console DevTools mà không ai mở lên xem). User báo 2 hiện tượng tưởng như không liên quan:

1. "Nghĩa tiếng Việt đọc sai" — phát âm nghe như đọc tiếng Anh phiên âm chữ Việt (do máy không có giọng **vi-VN**, engine fallback về giọng mặc định nhưng vẫn cố đọc text tiếng Việt).
2. "Chưa có đánh vần từng chữ" — bước đọc đánh vần (xếp SAU bước đọc nghĩa trong chuỗi `.then()`) **hoàn toàn không bao giờ chạy** — nguyên nhân thật: `TypeError: U.speakSpelled is not a function`, vì hàm `speakSpelled` được viết xong trong file `utils.js` nhưng **quên thêm vào object trả về ở cuối IIFE** (`return { el, $, speak, ... }` không có `speakSpelled`). Gọi `U.speakSpelled(...)` bên trong `.then()` ném lỗi → unhandled promise rejection → mọi `.then()` xếp SAU nó trong chain (kể cả đọc ví dụ) cũng biến mất theo, không có log nào lọt ra ngoài devtools console.

Nguyên nhân gốc rễ (Root Cause)

Hai lớp nguyên nhân cộng dồn, dễ nhầm lẫn với nhau khi debug:

(A) — nguyên nhân THẬT của "chưa có đánh vần": Module dùng pattern IIFE `window.U = (function () { ... return {a, b, c}; })()` — thêm 1 `function` mới bên trong IIFE **không tự động lộ ra ngoài**; phải

nhớ thêm tên hàm vào đúng object `return {...}` ở cuối file. Quên bước này là lỗi rất dễ mắc và rất dễ bỏ sót khi review (code bên trong file trông "đã có hàm", nhưng gọi từ file khác qua `U.tenHam` sẽ `undefined`).

(B) — nguyên nhân góp phần cho "nghĩa tiếng Việt đọc sai": `U.speak()` set `u.lang = opts.lang` tương minh (VD `'vi-VN'`) nhưng không kiểm tra máy có giọng khớp hay không (`pickVoice()` có thể trả `null`, `u.voice` không được gán) — trình duyệt vẫn cố phát bằng giọng mặc định khác ngôn ngữ, nghe như đọc sai/gần. `speak()` cũng không có timeout dự phòng nếu 1 utterance không hỗ trợ khiến `onend/onerror` không bắn — nên bất kỳ promise chain TTS nào phụ thuộc callback bên thứ 3 mà thiếu timeout đều có nguy cơ treo y hệt lỗi (A) dù nguyên nhân khác hẳn.

⚠ Bài học điều tra: khi thấy "bước sau trong 1 chuỗi `.then()` hoàn toàn không chạy, không lỗi gì cả với user" — **luôn mở DevTools Console trước khi suy đoán race-condition/hành vi trình duyệt** — 90% trường hợp là lỗi JS bình thường (typo, quên export, gọi nhầm tên) bị nuốt bởi unhandled promise rejection, không phải bug lạ của Web API.

```
function speak(text, opts) {
  return new Promise(resolve => {
    speechSynthesis.cancel();
    const u = new SpeechSynthesisUtterance(text);
    u.lang = opts.lang || 'en-US';
    const v = pickVoice(u.lang);
    if (v) u.voice = v;
    u.onend = resolve;
    u.onerror = resolve;
    speechSynthesis.speak(u);
  });
}
```

Code trên minh họa nguyên nhân (B) — không có timeout dự phòng khi `onend/onerror` không bắn. Đây là rủi ro chung của việc build promise chain phụ thuộc vào 1 sự kiện của API bên thứ 3 (OS/browser TTS engine), kể cả khi đã fix xong nguyên nhân (A) (quên export hàm).

Giải pháp

0. **Fix nguyên nhân THẬT trước tiên:** thêm tên hàm còn thiếu vào `return {...}` cuối IIFE (`return { ..., speakSpelled, hasVoice }`). Đây là 1 dòng nhưng là fix quyết định — mọi thứ khác bên dưới chỉ là hardening thêm.
1. Thêm **safety-net timeout** bên trong `speak()` — luôn `resolve()` sau tối đa vài giây dù `onend/onerror` không bắn, để bất kỳ chain nào phía sau luôn được tiếp tục chạy dù 1 bước bị "câm":

```
function speak(text, opts) {
  return new Promise(resolve => {
    speechSynthesis.cancel();
    const u = new SpeechSynthesisUtterance(text);
    // ... set lang/rate/voice ...
    let done = false;
    const finish = () => { if (done) return; done = true; resolve(); };
    u.onend = finish;
    u.onerror = finish;
    speechSynthesis.speak(u);
    setTimeout(finish, Math.max(4000, String(text).length * 200)); // lưới an toàn
  });
}
```

2. Thêm hàm `hasVoice(lang)` kiểm tra thật có giọng khớp prefix ngôn ngữ hay không (`voices.some(v => v.lang.startsWith(prefix))`), và **chỉ gọi TTS ngôn ngữ phụ (VD tiếng Việt) khi thực sự có giọng** — nếu không có, bỏ qua bước đó (im lặng) thay vì cố đọc bằng giọng sai (đọc sai nghe khó chịu hơn là không đọc).

Áp dụng lại (How to reuse)

- Ngay sau khi viết xong 1 `function` mới bên trong module IIFE (`window.X = (function() { ... })()`) → mở NGAY DevTools Console và gọi thử `X.tenHamMoi` trước khi coi task xong; đây là checklist 5 giây rẻ hơn rất nhiều so với việc debug theo hướng "chắc do trình duyệt/race condition".
- Khi thấy hiện tượng "bước sau trong 1 chuỗi `.then()` hoàn toàn không chạy, không lỗi hiển thị" → **mở Console trước tiên** để loại trừ lỗi JS bình thường (quên export, gọi sai tên) trước khi nghi ngờ hành vi lạ của Web API/race condition.
- Khi build bất kỳ promise chain nào phụ thuộc vào callback/event của **API trình duyệt hoặc SDK ngoài** (Web Speech, WebRTC, MediaRecorder, Bluetooth...) → **LUÔN** có timeout dự phòng để resolve, đừng tin tuyệt đối là event sẽ luôn bắn.
- Trước khi gọi TTS bằng 1 `lang` cụ thể khác ngôn ngữ chính của app (VD app tiếng Anh nhưng cần đọc mẫu tiếng Việt/Nhật/Trung xen kẽ) → kiểm tra `hasVoice()` trước, đừng giả định trình duyệt sẽ tự xử lý gracefully.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `pickVoice()` trả `null` không có nghĩa là utterance sẽ báo lỗi ngay — nó có thể phát bằng giọng mặc định (đọc sai) HOẶC treo vô thời hạn, tùy engine/OS — hành vi không nhất quán giữa Chrome/Edge/máy có cài ngôn ngữ khác nhau.
- ⚠ `speechSynthesis.cancel()` gọi ở đầu mỗi `speak()` để "dọn hàng đợi" trước khi đọc utterance mới — nếu gọi hàng loạt `speak()` liên tiếp không qua `.then()` (không đợi utterance trước xong), utterance trước có thể bị cắt ngang.
- ⚠ README của project này đã có sẵn pattern cảnh báo thiếu giọng cho tiếng Nhật/Trung (`lang.js` → `voiceWarning()`) — nên tái dùng pattern `hasVoice()` / cảnh báo này cho MỌI ngôn ngữ phụ được thêm sau, không riêng tiếng Việt.

Tham chiếu

- Project liên quan: EnglishTool (`app/assets/js/utils.js`, `app/assets/js/focus.js`)